Prokudin-Gorskii Görüntü İşleme Raporu

RAPOR BİLGİSİ

Ders adı: Computer Vision
Hazırlayan: Muhammed Ali Kandemir
Tarih: 05/03/2026

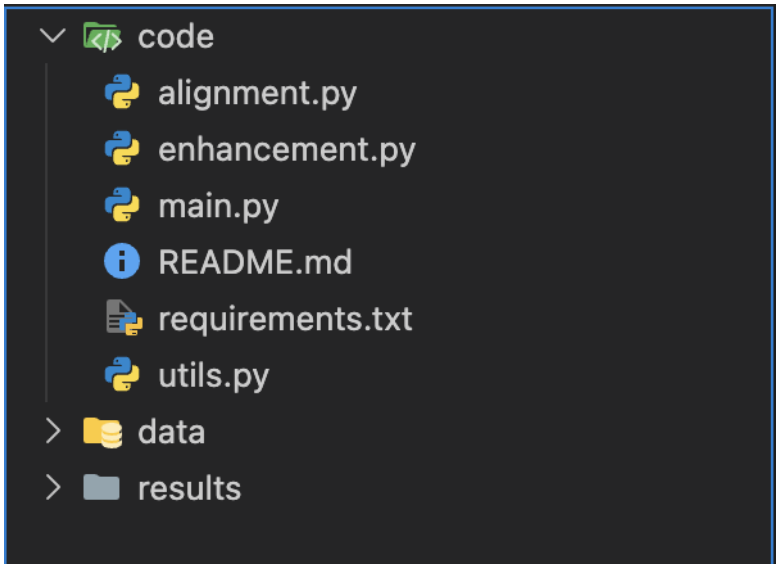
Anahtar kelimeler:
Bilgisayarla görme
Görüntü işleme
Prokudin-Gorskii

ÖZET

Bu raporda, tek kanallı tarihî görüntülerden renkli görüntü üretimi için uygulanan işlem hattı sunulmuştur. Çalışmada önce görüntü mavi, yeşil ve kırmızı kanallara ayrılmış, ardından kanallar arası kaymalar uygun benzerlik metrikleri kullanılarak hesaplanmıştır. Hesaplanan ofsetlerle kanallar hizalanmış ve birleşik renkli görüntü elde edilmiştir. Sonraki aşamada kontrast, parlaklık ve keskinlik iyileştirmeleri uygulanmış, kenar bölgelerde oluşan hizalama artefaktlarını azaltmak amacıyla kırpma işlemi gerçekleştirilmiştir. Elde edilen sonuçlar, hizalama kalitesinin özellikle doğru metrik seçimine duyarlı olduğunu ve son işleme adımlarının görsel algıyı belirgin biçimde artırdığını göstermektedir. Çalışma, temel bilgisayarlı görü yöntemleriyle düşük kaliteli veya ham görüntülerden daha anlamlı ve estetik çıktılar üretilebileceğini ortaya koymaktadır.

GİRİŞ

Proje genel olarak görüntüyü uygun formata çevirme, bölme, birleştirme ve düzeltme aşamalarından oluşmaktadır.



Şekil 1: klasör yapısı

Projenin çalıştırılması için temel komut python -u main.py –input ”path” şeklindedir. Burada input parametresi zorunludur; geçersiz veya eksik verildiğinde program işlemi durdurur. Sistem, input olarak hem tek bir dosya hem de bir klasör alabilecek şekilde tasarlanmıştır. Klasör verilmesi durumunda uygun uzantıdaki görüntüler otomatik olarak taranır ve toplu işleme yapılır.

Projede üç ek parametre bulunur: output, metric ve pyramid. output parametresi verilmezse sonuçlar varsayılan olarak proje içindeki results klasörüne yazılır; kullanıcı isterse farklı bir çıktı yolu belirleyebilir. metric parametresi kanal hizalama aşamasında kullanılan benzerlik yöntemini seçer ve ncc ile ssd seçeneklerini sunar. Varsayılan yöntem ncc'dir ve çoğu durumda daha kararlı sonuç üretir. pyramid parametresi arayüzde opsiyonel bir bayrak olarak yer alır ve genişletilebilir yapı için korunmuştur.

Uygulama akışı modüler bir şekilde ilerler. Önce görüntü yüklenir ve üç kanal bilgisi ayrıştırılır. Daha sonra hizalama modülü, mavi kanalı referans alarak yeşil ve kırmızı kanalların piksel kaymalarını hesaplar. Bu kaymalar uygulandıktan sonra renkli görüntü oluşturulur. Son aşamada iyileştirme modülü devreye girerek otomatik kontrast, gamma düzeltme, keskinleştirme ve kenar kırpma işlemleri uygular. Böylece hem hizalanmamış hem hizalanmış hem de iyileştirilmiş çıktılar üretilerek kullanıcıya karşılaştırmalı sonuç sunulur.

Bu proje yalnızca çıktı üretmekle kalmaz, aynı zamanda süreç görünürlüğü de sağlar. Çalışma sırasında bulunan kaymalar, skor değerleri ve işlem süreleri terminale yazdırılır; işlem sonunda özet tablo verilir. Bu yaklaşım, hem akademik raporlama hem de teknik doğrulama açısından avantaj sağlar. Kısacası proje, ham görüntüden nihai renkli çıktıya kadar olan süreci açık, izlenebilir ve geliştirilebilir bir mimariyle gerçekleştirmektedir.

main.py

main.py projenin ana kontrol dosyasıdır ve tüm akışı burası yönetir. Program çalışınca önce komut satırından input, output, metric ve pyramid parametrelerini alır. Input yolu kontrol edilir; yol yoksa veya dosya/klasör değilse hata verip durur. Input bir dosyaysa tek dosya, klasörse içindeki jpg ve tif dosyaları işleme listesine eklenir. Output yolu için de doğrulama yapılır; geçersiz bir yol verilirse kullanıcıya varsayılan results klasörüne dönmek isteyip istemediği sorulur.

Her dosya için processimage fonksiyonu çağrılır. Bu fonksiyon görüntüyü yükler, tek görüntüyü üç parçaya ayırarak mavi-yeşil-kırmızı kanalları çıkarır. Sonra hizalama modülüyle yeşil ve kırmızı kanalların maviye göre kaymasını hesaplar ve bu kaymaları uygular. Böylece hizalanmış renkli görüntü elde edilir. Ardından enhancement adımıyla kontrast, gamma ve keskinlik iyileştirmesi yapılır; kenar kırpma ile hizalama sonrası oluşan renkli sınırlar temizlenir. Son olarak unaligned, aligned ve enhanced olmak üzere üç çıktı dosyası kaydedilir. main.py ayrıca kullanıcıya süreç bilgisini de verir. Her görsel için bulunan kaymalar, skorlar ve işlem süresi yazdırılır. Tüm dosyalar tamamlandığında da sonuçlar özet tablo halinde ekrana basılır. Yani bu dosya hem giriş doğrulamasını yapar hem işleme modüllerini sıralı çalıştırır hem de çıktıları raporlar.

Şekil 2: Unaligned image

Şekil 3: Aligned image

Şekil 4: Enhanced image

alignment.py

alignment.py dosyası, projedeki en kritik adımı yani kanal hizalamayı gerçekleştirir. Bu projede giriş görüntüsü aslında tek bir gri görüntüdür ama içinde üst üste dizilmiş üç ayrı kanal bilgisi bulunur. split işleminden sonra bu kanallar mavi, yeşil ve kırmızı olarak ayrılır. Sorun şu ki bu kanallar çekim sırasında tam aynı noktaya denk gelmez; aralarında birkaç piksel kayma olur. alignment modülünün amacı bu kaymayı otomatik olarak bulup düzeltmektir.

Modülün merkezinde **`align_channels`** fonksiyonu bulunur. Bu fonksiyon iki görüntü alır: biri referans kanal, diğeri hizalanacak hedef kanal. Kodda referans olarak genellikle mavi kanal kullanılır, yeşil ve kırmızı kanallar buna göre hizalanır. Fonksiyon önce kenar kırpma uygular. Bunun sebebi, kaydırma işlemi sırasında özellikle sınır bölgelerde sahte benzerlik oluşabilmesidir. Kenarları dışarıda bırakmak, metrik hesaplarını daha güvenilir hale getirir.

Sonra algoritma brute-force tarama yapar. Yani belirlenen aralıkta tüm dx ve dy kombinasyonlarını tek tek dener. Her kombinasyonda hedef kanal np.roll ile kaydırılır. Kaydırılmış görüntü ile referans görüntünün kırpılmış bölgeleri karşılaştırılır ve bir skor üretilir. Eğer metrik ncc ise amaç skoru maksimum yapmak, ssd ise amaç skoru minimum yapmaktır. Fonksiyon denenen tüm kaydırmalar içinde en iyi sonucu veren dx ve dy değerlerini saklar. Tarama bittiğinde en iyi kayma ve o kaymaya ait skor döndürülür.

Bulunan kayma **`apply_alignment`** fonksiyonu ile gerçek kanala uygulanır. Bu fonksiyon karmaşık bir işlem yapmaz; sadece np.roll ile kanalı satır ve sütun eksenlerinde taşır. Ana akışta bu adım hem yeşil hem kırmızı kanal için tekrarlandığı için iki kanal da maviye hizalanmış olur. Sonraki aşamada kanallar birleştirildiğinde renk saçılması azalır, kenarlar daha düzgün görünür ve görüntüdeki detaylar daha net ortaya çıkar.

enhancement.py

enhancement.py dosyası, hizalanmış görüntünün görsel kalitesini artırmak için kullanılan son işleme modülüdür. Bu aşama hizalamadan sonra devreye girer; yani önce kanallar doğru konuma getirilir, sonra görüntü daha okunabilir ve estetik hale getirilir. Modülün amacı yeni bilgi üretmek değil, mevcut bilgiyi daha net ve dengeli göstermektir. Bu yüzden kontrast, ton dağılımı, keskinlik ve kenar temizliği gibi klasik iyileştirme adımları ardışık şekilde uygulanır.

İlk adım **`auto_contrast`** fonksiyonudur. Bu fonksiyon her renk kanalını ayrı ayrı işler ve düşük-yüksek yoğunluk uçlarını yüzdelik dilime göre kırpar. Yani çok karanlık ve çok parlak uç değerlerin etkisini azaltarak orta bölgelerdeki detayları görünür hale getirir. Kodda percentile=2 kullanıldığı için her kanalın en alt yüzde 2 ve en üst yüzde 2 kısmı kırpılarak yeni aralığa gerilir. Sonuç olarak kontrast artar ve soluk bölgeler daha belirgin olur.

İkinci adım **`gamma_correction`** fonksiyonudur. Gamma düzeltme, görüntünün ton eğrisini doğrusal olmayan şekilde yeniden dağıtır. Kodda gamma=1.2 seçildiği için özellikle orta parlaklık seviyelerinde görsel denge iyileşir. Uygulama bir lookup table üzerinden yapıldığı için hesaplama hızlıdır. Bu adım, sadece parlaklığı artırmak değil, ton geçişlerini daha doğal hale getirmek için kullanılır.

Üçüncü adım **`sharpen_image`** fonksiyonudur. Burada görüntünün Gauss bulanık versiyonu üretilir, sonra orijinal görüntü ile ağırlıklı farkı alınarak unsharp mask benzeri bir keskinleştirme yapılır. Kodda amount=0.5 olduğu için keskinleştirme agresif değildir; amaç kenarları belirginleştirmek ama gürültüyü aşırı yükseltmemektir. Bu adım özellikle yapıların sınırlarını daha okunur hale getirir.

Bu üç adımı birleştiren fonksiyon **`enhance_image`** fonksiyonudur. Yani pipeline sırası sabittir: önce auto contrast, sonra gamma correction, sonra sharpen. Sıralama önemlidir çünkü kontrast düzeltilmeden yapılan keskinleştirme gürültüyü büyütebilir; gamma ayarı ise keskinleştirmeden önce tonal dağılımı dengeler. Bu nedenle modül sadece tek bir filtre değil, kontrollü bir iyileştirme zinciri sunar.

Son adım **`crop_edges`** fonksiyonudur. Hizalama sonrası bazı görüntülerde kenarlarda renkli bantlar veya boşluk kaynaklı artefaktlar oluşur. Bu fonksiyon görüntünün dört kenarından belirli bir yüzde kadar kırpar ve bu problemli bölgeleri temizler. Kodda `crop_percent=0.075` kullanıldığı için her kenardan yaklaşık yüzde 7.5 çıkarılır. Böylece nihai çıktı daha temiz görünür ve hizalama izleri büyük ölçüde kaldırılır.

utils.py

utils.py dosyası, projenin temel yardımcı fonksiyonlarını taşıyan altyapı modülüdür. Ana akışta kullanılan kritik işlemler burada toplandığı için kodun geri kalanı sade kalır. Modülün ana görevleri; görüntüyü yüklemek, kanallara ayırmak, benzerlik metriği hesaplamak ve renkli görüntüyü birleştirmektir.

İlk adımda **`load_image`** giriş dosyasını okur, dosya açılamazsa hata verir, ardından görüntüyü uygun formata çevirip gri görüntü olarak döndürür. Bu fonksiyon, tüm işlem hattı için standart başlangıç verisini üretir.

İkinci adımda **`split_image`** gri görüntüyü yükseklik boyunca üç eşit parçaya ayırır ve mavi-yeşil-kırmızı kanal katmanlarını çıkarır. Bu ayrıştırma doğru yapılmazsa hizalama zinciri doğru sonuç vermez.

Benzerlik hesaplarında iki metrik vardır: **`ssd_metric`** ve **`ncc_metric`**. **`ssd_metric`** farkın karesel toplamını verir ve düşük değer daha iyi eşleşmeyi ifade eder.

`ncc_metric` normalize korelasyon üretir ve yüksek değer daha iyi eşleşme anlamına gelir. Her iki fonksiyon da boyut kontrolü yaparak güvenli hesaplama sağlar.

Son olarak **`create_color_image`** hizalanmış kanalları birleştirip tek bir renkli görüntü üretir. Böylece hizalama sonrası oluşan nihai görüntü kaydedilebilir hale gelir.

Tablo 1: Sonuç Tablosu

Görüntü	G shift	R shift	Süre (s)
00757v	(3, 2)	(5, 5)	0.73
31421v	(0, 8)	(0, 13)	0.71
00106v	(1, 4)	(-1, 9)	0.67
01598v	(0, 8)	(-1, 15)	0.63
01167v	(0, 5)	(-2, 12)	0.68
10131v	(2, 6)	(3, 12)	0.62
00804v	(-2, 6)	(-4, 13)	0.63
00458v	(1, 4)	(3, 9)	0.67
00194v	(2, 4)	(4, 7)	0.64
01269v	(2, 6)	(3, 13)	0.67
01880v	(2, 6)	(4, 14)	0.65
00106v (2)	(1, 4)	(-1, 9)	0.65
00907v	(0, 2)	(0, 6)	0.65
00911v	(-1, 1)	(-1, 13)	0.65
00106v (3)	(1, 4)	(-1, 9)	0.71
00398v	(3, 5)	(4, 11)	0.62
01031v	(1, 1)	(2, 4)	0.66
00163v	(1, -3)	(1, -4)	0.59
00088v	(3, 3)	(5, 4)	0.66
00600v	(4, 6)	(6, 13)	0.69
01728v	(1, 8)	(1, 15)	0.72
00106v (1)	(1, 4)	(-1, 9)	0.63
00153v	(3, 7)	(5, 14)	0.68
01522v	(2, 6)	(2, 13)	0.62
01164v	(2, 6)	(3, 11)	0.67
00889v	(2, 2)	(3, 4)	0.73
00888v	(1, 6)	(1, 12)	0.63
00056v	(1, 6)	(1, 13)	0.67
01597v	(1, 7)	(1, 15)	0.67

Program sonunda ekrana basılan özet tablo, işlenen tüm görüntülerin teknik sonuçlarını tek yerde gösteren kısa bir raportur. Bu tabloda her satır bir görüntüyü temsil eder ve ilgili görüntü için bulunan hizalama değerleri ile işlem süresi birlikte verilir. “Görüntü” sütunu dosya adını, “G Shift” sütunu yeşil kanalın mavi kanala göre kaç piksel kaydırıldığını, “R Shift” sütunu ise kırmızı kanalın mavi kanala göre kaydırma miktarını gösterir. “Süre (s)” sütunu da o görüntünün kaç saniyede işlendiğini belirtir. Bu yapı sayesinde kullanıcı, hangi görüntüde hangi kaydırmanın bulunduğunu, sonuçların tutarlı olup olmadığını ve performansın nasıl değiştiğini hızlıca değerlendirebilir. Özellikle farklı metriklerle deneme yapıldığında, bu tablo karşılaştırma ve doğrulama için doğrudan kullanılabilecek net bir özet sunar.

Şekil 5: result

Şekil 6: result

Şekil 7: result